编程作业 2

2025 年 5 月 7 日

0.1 题目一：MNIST 手写字符识别程序阅读题

任务描述：

阅读并运行以下的 Keras 代码（基于 MNIST 数据集），回答相关问题。

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.utils import to_categorical
from keras.optimizers import Adam

import matplotlib
matplotlib.rc("font",family='SimHei') # 中文字体    /home/google/anaconda3/envs/
 ↪kerasMO/lib/python3.7/site-packages/matplotlib/mpl-data/fonts/ttf
plt.rcParams['axes.unicode_minus']=False  # 用来正常显示负号



# 加载 MNIST 数据集
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 数据归一化 (0-255 -> 0-1)
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# 将标签转换为 one-hot 编码
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)

# 查看数据形状
print("训练数据形状:", x_train.shape)  # (60000, 28, 28)
print("训练标签形状:", y_train.shape)  # (60000, 10)


model = Sequential()  # 创建空模型

model.add(Flatten(input_shape=(28, 28)))  # 展平层，将 28x28 图像转为 784 维向量
model.add(Dense(16, activation='relu'))    # 第一个隐藏层，16 个神经元
model.add(Dense(16, activation='relu'))    # 第二个隐藏层，16 个神经元
model.add(Dense(10, activation='softmax')) # 输出层，10 个神经元对应 10 个数字类别


# 编译模型
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# 查看模型摘要
model.summary()

history = model.fit(x_train, y_train,
                    batch_size=32,
                    epochs=10,
                    validation_split=0.2,  # 使用 20% 训练数据作为验证集
                    verbose=1)

# 在测试集上评估
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\n测试集准确率: {test_acc:.4f}")

# 绘制训练曲线
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
```

```
plt.plot(history.history['accuracy'], label='训练准确率')
plt.plot(history.history['val_accuracy'], label='验证准确率')
plt.title('准确率曲线')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='训练损失')
plt.plot(history.history['val_loss'], label='验证损失')
plt.title('损失曲线')
plt.legend()
plt.show()
```

```
2025-05-07 15:57:33.097214: W
tensorflow/stream_executor/platform/default/dso_loader.cc:59] Could not load
dynamic library 'libcudart.so.10.1'; dlerror: libcudart.so.10.1: cannot open
shared object file: No such file or directory
2025-05-07 15:57:33.097249: I tensorflow/stream_executor/cuda/cudart_stub.cc:29]
Ignore above cudart dlerror if you do not have a GPU set up on your machine.

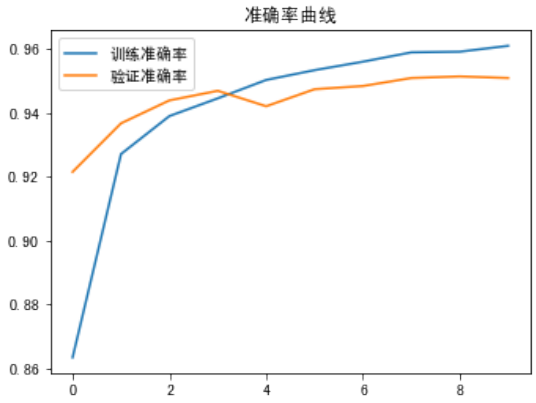
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11493376/11490434 [==============================] - 2s 0us/step
训练数据形状: (60000, 28, 28)
训练标签形状: (60000, 10)
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 16)                12560
_________________________________________________________________
dense_1 (Dense)              (None, 16)                272
_________________________________________________________________
dense_2 (Dense)              (None, 10)                170
=================================================================
Total params: 13,002
```

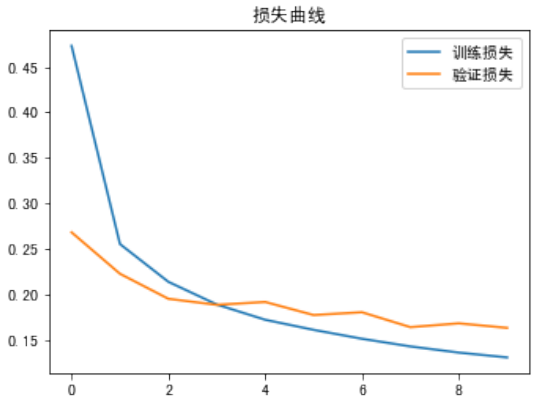
```
Trainable params: 13,002
Non-trainable params: 0
_________________________________________________________________
2025-05-07 15:57:38.293573: W
tensorflow/stream_executor/platform/default/dso_loader.cc:59] Could not load
dynamic library 'libcuda.so.1'; dlerror: libcuda.so.1: cannot open shared object
file: No such file or directory
2025-05-07 15:57:38.293631: W
tensorflow/stream_executor/cuda/cuda_driver.cc:312] failed call to cuInit:
UNKNOWN ERROR (303)
2025-05-07 15:57:38.293700: I
tensorflow/stream_executor/cuda/cuda_diagnostics.cc:163] no NVIDIA GPU device is
present: /dev/nvidia0 does not exist
2025-05-07 15:57:38.294672: I tensorflow/core/platform/cpu_feature_guard.cc:142]
This TensorFlow binary is optimized with oneAPI Deep Neural Network Library
(oneDNN)to use the following CPU instructions in performance-critical
operations:  AVX2 AVX512F FMA
To enable them in other operations, rebuild TensorFlow with the appropriate
compiler flags.
2025-05-07 15:57:38.328330: I
tensorflow/core/platform/profile_utils/cpu_utils.cc:104] CPU Frequency:
2300000000 Hz
2025-05-07 15:57:38.334130: I tensorflow/compiler/xla/service/service.cc:168]
XLA service 0x1735c60 initialized for platform Host (this does not guarantee
that XLA will be used). Devices:
2025-05-07 15:57:38.334164: I tensorflow/compiler/xla/service/service.cc:176]
StreamExecutor device (0): Host, Default Version

Epoch 1/10
1500/1500 [==============================] - 12s 8ms/step - loss: 0.4731 -
accuracy: 0.8635 - val_loss: 0.2683 - val_accuracy: 0.9215
Epoch 2/10
1500/1500 [==============================] - 12s 8ms/step - loss: 0.2555 -
accuracy: 0.9271 - val_loss: 0.2228 - val_accuracy: 0.9367
Epoch 3/10
1500/1500 [==============================] - 11s 7ms/step - loss: 0.2140 -
accuracy: 0.9389 - val_loss: 0.1954 - val_accuracy: 0.9438
```

```
Epoch 4/10
1500/1500 [==============================] - 11s 7ms/step - loss: 0.1891 -
accuracy: 0.9445 - val_loss: 0.1887 - val_accuracy: 0.9468
Epoch 5/10
1500/1500 [==============================] - 11s 7ms/step - loss: 0.1723 -
accuracy: 0.9502 - val_loss: 0.1919 - val_accuracy: 0.9420
Epoch 6/10
1500/1500 [==============================] - 11s 7ms/step - loss: 0.1613 -
accuracy: 0.9533 - val_loss: 0.1775 - val_accuracy: 0.9473
Epoch 7/10
1500/1500 [==============================] - 11s 8ms/step - loss: 0.1515 -
accuracy: 0.9559 - val_loss: 0.1806 - val_accuracy: 0.9483
Epoch 8/10
1500/1500 [==============================] - 12s 8ms/step - loss: 0.1431 -
accuracy: 0.9589 - val_loss: 0.1642 - val_accuracy: 0.9508
Epoch 9/10
1500/1500 [==============================] - 12s 8ms/step - loss: 0.1363 -
accuracy: 0.9591 - val_loss: 0.1684 - val_accuracy: 0.9513
Epoch 10/10
1500/1500 [==============================] - 12s 8ms/step - loss: 0.1310 -
accuracy: 0.9609 - val_loss: 0.1634 - val_accuracy: 0.9508
```

测试集准确率：0.9560





0.1.1 提问点：

1. 为什么要将图像数据从 (28, 28) 重塑为 (28*28,)？ 答：全连接网络的第一层（即输入层）一般是个向量，因此对手写体图片这样的二维数组来说，需要先把它 “拉直” 成**一个向量**，比如一张 28×28 的图片，重塑为（28*28,），因此被转换成 784×1 的向量。

2. 归一化到 [0, 1]（除以 255）的作用是什么？ 答：灰度有 256 个等级，除以 255 将灰度归一化到【0,1】可以**减小输入数据的尺度**，归一化可以将不同特征的取值范围调整到相似的区间，这样在梯度下降过程中，每个特征对应的权重更新步长相对比较一致，每个特征对损失函数的**影响相对平衡**，权重更新更加平稳，从而加速了神经网络的**收敛速度**。同时，归一化可以将输入数据的取值范围控制在一个合理的区间内，使得**激活函数**等操作能够在其**有效工作范围**内进行计算。

3. 输出层的激活函数为什么用 softmax？若改为 sigmoid 会怎样？ 答：1. 神经网络做分类问题时常用 softmax，Softmax 函数往往被放到神经网络的最后一层，直接获得分类结果的输出，可以将输出转换为**概率分布**。2. 若改为 sigmoid ，尽管 sigmoid 在任何位置可导，然而它会**导致梯度消失**问题，因此不利于神经网络学习。

4. 解释 categorical_crossentropy 损失函数的意义。 答：loss=’categorical_crossentropy’，该损失函数常用于**多分类任务**，是基于交叉熵（cross-entropy）的损失函数。交叉熵衡量的是两个概率分布之间的差异，在多分类问题中，它衡量的是模型输出的**概率分布和真实标签**的概率分布之间的差异。由于使用 one-hot，在交叉熵损失函数中只有**真实类别对应**的那一项会被计算在内其他类别的项在求和过程中均为 0，多分类问题的交叉熵损失，只与真实类别对应的模型预测概率有关，即便**模型对其他类别的预测概率不准确但只要对真实类别的预测概率较高损失函数的值仍然较低**。

0.2 题目二：Boston 房价非线性回归程序阅读题

任务描述：

阅读并运行以下的 Boston 房价预测代码，回答相关问题。

[2]:
```
import numpy as np
from keras.datasets import boston_housing
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam
from sklearn.preprocessing import StandardScaler
```

```
import matplotlib.pyplot as plt

# 1. 加载数据集
(train_data, train_targets), (test_data, test_targets) = boston_housing.
 ↪load_data()

# 2. 数据标准化（重要步骤!）
scaler = StandardScaler()
train_data = scaler.fit_transform(train_data)
test_data = scaler.transform(test_data)

# 3. 构建全连接神经网络模型
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(train_data.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(1))  # 输出层不使用激活函数，因为这是回归问题

# 4. 编译模型
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='mse',  # 均方误差适合回归问题
              metrics=['mae'])  # 平均绝对误差

# 打印模型结构
model.summary()

# 5. 训练模型
history = model.fit(train_data, train_targets,
                    epochs=100,
                    batch_size=16,
                    validation_split=0.2,
                    verbose=1)

# 6. 评估模型
test_mse, test_mae = model.evaluate(test_data, test_targets, verbose=0)
print(f"\n测试集 MSE: {test_mse:.4f}")
print(f"测试集 MAE: {test_mae:.4f} (约 ${test_mae*1000:.0f})")
```

```
# 7. 绘制训练曲线
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Validation MAE')
plt.title('MAE over Epochs')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss (MSE)')
plt.plot(history.history['val_loss'], label='Validation Loss (MSE)')
plt.title('MSE over Epochs')
plt.xlabel('Epoch')
plt.ylabel('MSE')
plt.legend()

plt.tight_layout()
plt.show()

# 8. 进行预测示例
sample_index = 10
sample_house = test_data[sample_index]
actual_price = test_targets[sample_index]
predicted_price = model.predict(np.array([sample_house]))[0][0]

print(f"\n示例房屋预测:")
print(f"真实价格: ${actual_price*1000:.0f}")
print(f"预测价格: ${predicted_price*1000:.0f}")
print(f"误差: ${abs(actual_price-predicted_price)*1000:.0f}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/boston_housing.npz
57344/57026 [==============================] - 0s 7us/step
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_3 (Dense)              (None, 64)                896
_________________________________________________________________
dense_4 (Dense)              (None, 64)                4160
_________________________________________________________________
dense_5 (Dense)              (None, 1)                 65
=================================================================
Total params: 5,121
Trainable params: 5,121
Non-trainable params: 0
_________________________________________________________________
Epoch 1/100
21/21 [==============================] - 0s 16ms/step - loss: 527.0845 - mae:
20.9892 - val_loss: 546.8517 - val_mae: 21.4843
Epoch 2/100
21/21 [==============================] - 0s 9ms/step - loss: 428.8368 - mae:
18.5216 - val_loss: 424.6302 - val_mae: 18.4937
Epoch 3/100
21/21 [==============================] - 0s 10ms/step - loss: 290.5071 - mae:
14.7578 - val_loss: 248.5711 - val_mae: 13.5393
Epoch 4/100
21/21 [==============================] - 0s 10ms/step - loss: 138.9711 - mae:
9.5792 - val_loss: 102.3634 - val_mae: 7.7442
Epoch 5/100
21/21 [==============================] - 0s 13ms/step - loss: 62.4458 - mae:
5.9839 - val_loss: 63.9012 - val_mae: 6.1133
Epoch 6/100
21/21 [==============================] - 0s 10ms/step - loss: 43.0507 - mae:
4.6606 - val_loss: 46.6490 - val_mae: 5.1951
Epoch 7/100
21/21 [==============================] - 0s 13ms/step - loss: 32.5011 - mae:
3.9421 - val_loss: 36.3318 - val_mae: 4.6058
Epoch 8/100
21/21 [==============================] - 0s 10ms/step - loss: 27.4059 - mae:
```

```
3.5977 - val_loss: 30.6463 - val_mae: 4.2503
Epoch 9/100
21/21 [==============================] - 0s 23ms/step - loss: 24.5645 - mae:
3.3546 - val_loss: 28.0714 - val_mae: 4.0330
Epoch 10/100
21/21 [==============================] - 0s 10ms/step - loss: 22.6458 - mae:
3.2235 - val_loss: 25.4662 - val_mae: 3.8830
Epoch 11/100
21/21 [==============================] - 0s 13ms/step - loss: 21.0178 - mae:
3.1147 - val_loss: 23.9289 - val_mae: 3.7915
Epoch 12/100
21/21 [==============================] - 0s 10ms/step - loss: 19.7715 - mae:
3.0401 - val_loss: 22.5767 - val_mae: 3.6791
Epoch 13/100
21/21 [==============================] - 0s 10ms/step - loss: 18.7213 - mae:
2.9773 - val_loss: 21.8853 - val_mae: 3.6062
Epoch 14/100
21/21 [==============================] - 0s 10ms/step - loss: 17.6894 - mae:
2.8445 - val_loss: 20.7472 - val_mae: 3.4944
Epoch 15/100
21/21 [==============================] - 0s 12ms/step - loss: 16.7684 - mae:
2.7818 - val_loss: 20.1184 - val_mae: 3.4350
Epoch 16/100
21/21 [==============================] - 0s 10ms/step - loss: 16.2032 - mae:
2.7610 - val_loss: 19.6615 - val_mae: 3.3705
Epoch 17/100
21/21 [==============================] - 0s 13ms/step - loss: 15.4804 - mae:
2.7011 - val_loss: 19.0923 - val_mae: 3.3098
Epoch 18/100
21/21 [==============================] - 0s 11ms/step - loss: 14.8077 - mae:
2.5921 - val_loss: 18.3916 - val_mae: 3.2282
Epoch 19/100
21/21 [==============================] - 0s 12ms/step - loss: 14.1738 - mae:
2.5683 - val_loss: 17.8523 - val_mae: 3.1379
Epoch 20/100
21/21 [==============================] - 0s 10ms/step - loss: 13.7643 - mae:
2.5055 - val_loss: 17.3044 - val_mae: 3.1043
```

```
Epoch 21/100
21/21 [==============================] - 0s 10ms/step - loss: 13.2920 - mae:
2.4963 - val_loss: 17.1976 - val_mae: 3.0530
Epoch 22/100
21/21 [==============================] - 0s 13ms/step - loss: 12.8430 - mae:
2.4322 - val_loss: 16.6514 - val_mae: 3.0191
Epoch 23/100
21/21 [==============================] - 0s 11ms/step - loss: 12.5542 - mae:
2.4070 - val_loss: 16.6931 - val_mae: 2.9763
Epoch 24/100
21/21 [==============================] - 0s 14ms/step - loss: 12.2280 - mae:
2.4051 - val_loss: 16.5654 - val_mae: 3.0245
Epoch 25/100
21/21 [==============================] - 0s 13ms/step - loss: 11.7256 - mae:
2.3225 - val_loss: 16.1226 - val_mae: 2.9696
Epoch 26/100
21/21 [==============================] - 0s 10ms/step - loss: 11.7303 - mae:
2.3778 - val_loss: 16.1176 - val_mae: 2.9665
Epoch 27/100
21/21 [==============================] - 0s 14ms/step - loss: 11.4127 - mae:
2.2903 - val_loss: 16.0202 - val_mae: 2.9553
Epoch 28/100
21/21 [==============================] - 0s 10ms/step - loss: 11.2297 - mae:
2.2834 - val_loss: 15.3763 - val_mae: 2.8988
Epoch 29/100
21/21 [==============================] - 0s 12ms/step - loss: 10.7553 - mae:
2.2549 - val_loss: 15.5394 - val_mae: 2.8617
Epoch 30/100
21/21 [==============================] - 0s 10ms/step - loss: 10.6866 - mae:
2.2331 - val_loss: 15.2388 - val_mae: 2.8529
Epoch 31/100
21/21 [==============================] - 0s 10ms/step - loss: 10.3331 - mae:
2.2245 - val_loss: 15.2839 - val_mae: 2.8582
Epoch 32/100
21/21 [==============================] - 0s 10ms/step - loss: 10.1647 - mae:
2.1937 - val_loss: 14.9415 - val_mae: 2.8239
Epoch 33/100
```

```
21/21 [==============================] - 0s 10ms/step - loss: 10.0003 - mae:
2.1671 - val_loss: 15.0276 - val_mae: 2.8089
Epoch 34/100
21/21 [==============================] - 0s 9ms/step - loss: 10.0386 - mae:
2.1647 - val_loss: 14.5995 - val_mae: 2.7439
Epoch 35/100
21/21 [==============================] - 0s 10ms/step - loss: 9.7674 - mae:
2.1638 - val_loss: 14.9727 - val_mae: 2.7779
Epoch 36/100
21/21 [==============================] - 0s 20ms/step - loss: 9.7500 - mae:
2.1847 - val_loss: 14.5209 - val_mae: 2.7703
Epoch 37/100
21/21 [==============================] - 0s 10ms/step - loss: 9.5678 - mae:
2.1188 - val_loss: 15.2192 - val_mae: 2.7421
Epoch 38/100
21/21 [==============================] - 0s 13ms/step - loss: 9.3237 - mae:
2.1350 - val_loss: 14.9270 - val_mae: 2.7338
Epoch 39/100
21/21 [==============================] - 0s 9ms/step - loss: 9.1910 - mae:
2.1161 - val_loss: 14.8055 - val_mae: 2.7547
Epoch 40/100
21/21 [==============================] - 0s 10ms/step - loss: 9.4994 - mae:
2.1177 - val_loss: 14.7528 - val_mae: 2.7034
Epoch 41/100
21/21 [==============================] - 0s 9ms/step - loss: 9.1388 - mae:
2.1099 - val_loss: 14.3860 - val_mae: 2.7046
Epoch 42/100
21/21 [==============================] - 0s 14ms/step - loss: 9.0927 - mae:
2.0974 - val_loss: 14.4637 - val_mae: 2.7238
Epoch 43/100
21/21 [==============================] - 0s 9ms/step - loss: 9.1091 - mae:
2.0640 - val_loss: 14.5823 - val_mae: 2.6712
Epoch 44/100
21/21 [==============================] - 0s 14ms/step - loss: 8.8387 - mae:
2.0399 - val_loss: 14.2868 - val_mae: 2.6392
Epoch 45/100
21/21 [==============================] - 0s 14ms/step - loss: 8.9719 - mae:
```

```
2.1034 - val_loss: 14.3678 - val_mae: 2.6824
Epoch 46/100
21/21 [==============================] - 0s 9ms/step - loss: 8.6819 - mae:
2.0275 - val_loss: 15.2363 - val_mae: 2.6650
Epoch 47/100
21/21 [==============================] - 0s 11ms/step - loss: 8.6085 - mae:
2.0235 - val_loss: 14.4963 - val_mae: 2.6392
Epoch 48/100
21/21 [==============================] - 0s 13ms/step - loss: 8.4038 - mae:
2.0111 - val_loss: 14.9305 - val_mae: 2.6502
Epoch 49/100
21/21 [==============================] - 0s 15ms/step - loss: 8.3451 - mae:
2.0421 - val_loss: 14.7647 - val_mae: 2.6435
Epoch 50/100
21/21 [==============================] - 0s 9ms/step - loss: 8.2169 - mae:
1.9798 - val_loss: 14.3607 - val_mae: 2.6302
Epoch 51/100
21/21 [==============================] - 0s 13ms/step - loss: 8.0798 - mae:
1.9831 - val_loss: 14.9834 - val_mae: 2.6273
Epoch 52/100
21/21 [==============================] - 0s 10ms/step - loss: 8.0550 - mae:
2.0005 - val_loss: 14.5992 - val_mae: 2.6665
Epoch 53/100
21/21 [==============================] - 0s 11ms/step - loss: 8.1161 - mae:
1.9746 - val_loss: 14.8786 - val_mae: 2.6195
Epoch 54/100
21/21 [==============================] - 0s 9ms/step - loss: 8.0696 - mae:
2.0048 - val_loss: 14.2941 - val_mae: 2.6360
Epoch 55/100
21/21 [==============================] - 0s 10ms/step - loss: 7.9665 - mae:
1.9403 - val_loss: 15.0082 - val_mae: 2.6306
Epoch 56/100
21/21 [==============================] - 0s 13ms/step - loss: 7.9625 - mae:
1.9801 - val_loss: 14.9868 - val_mae: 2.6738
Epoch 57/100
21/21 [==============================] - 0s 9ms/step - loss: 7.9729 - mae:
1.9602 - val_loss: 14.8857 - val_mae: 2.6097
```

```
Epoch 58/100
21/21 [==============================] - 0s 9ms/step - loss: 7.9410 - mae:
1.9796 - val_loss: 14.5013 - val_mae: 2.6004
Epoch 59/100
21/21 [==============================] - 0s 10ms/step - loss: 8.1002 - mae:
1.9630 - val_loss: 14.8066 - val_mae: 2.6262
Epoch 60/100
21/21 [==============================] - 0s 10ms/step - loss: 7.7336 - mae:
1.9540 - val_loss: 14.9834 - val_mae: 2.6174
Epoch 61/100
21/21 [==============================] - 0s 13ms/step - loss: 7.9185 - mae:
2.0106 - val_loss: 14.3325 - val_mae: 2.6553
Epoch 62/100
21/21 [==============================] - 0s 11ms/step - loss: 7.6673 - mae:
1.9199 - val_loss: 15.0125 - val_mae: 2.6057
Epoch 63/100
21/21 [==============================] - 0s 14ms/step - loss: 7.4776 - mae:
1.9009 - val_loss: 14.4947 - val_mae: 2.5723
Epoch 64/100
21/21 [==============================] - 0s 9ms/step - loss: 7.3574 - mae:
1.8828 - val_loss: 15.2407 - val_mae: 2.6290
Epoch 65/100
21/21 [==============================] - 0s 13ms/step - loss: 7.3877 - mae:
1.9176 - val_loss: 14.6609 - val_mae: 2.5973
Epoch 66/100
21/21 [==============================] - 0s 10ms/step - loss: 7.6204 - mae:
1.9126 - val_loss: 14.4034 - val_mae: 2.5638
Epoch 67/100
21/21 [==============================] - 0s 10ms/step - loss: 7.2866 - mae:
1.9080 - val_loss: 15.2849 - val_mae: 2.6404
Epoch 68/100
21/21 [==============================] - 0s 13ms/step - loss: 7.1901 - mae:
1.8675 - val_loss: 14.3847 - val_mae: 2.5663
Epoch 69/100
21/21 [==============================] - 0s 9ms/step - loss: 7.1378 - mae:
1.8664 - val_loss: 14.7858 - val_mae: 2.6196
Epoch 70/100
```

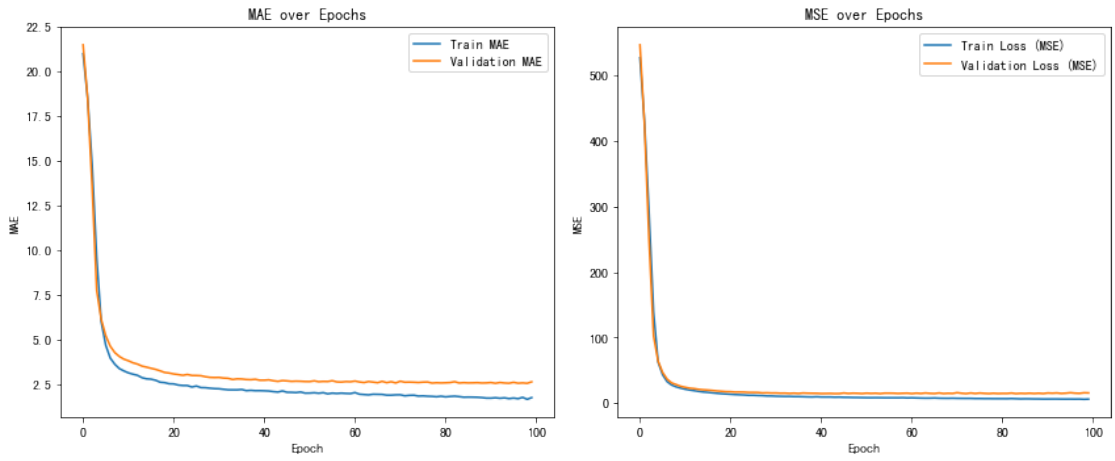
```
21/21 [==============================] - 0s 9ms/step - loss: 7.2771 - mae:
1.8835 - val_loss: 14.5518 - val_mae: 2.5502
Epoch 71/100
21/21 [==============================] - 0s 10ms/step - loss: 7.0881 - mae:
1.8876 - val_loss: 15.7604 - val_mae: 2.6474
Epoch 72/100
21/21 [==============================] - 0s 9ms/step - loss: 7.0274 - mae:
1.8295 - val_loss: 14.8153 - val_mae: 2.5909
Epoch 73/100
21/21 [==============================] - 0s 10ms/step - loss: 6.9814 - mae:
1.8567 - val_loss: 14.3119 - val_mae: 2.5974
Epoch 74/100
21/21 [==============================] - 0s 13ms/step - loss: 6.9984 - mae:
1.8638 - val_loss: 15.2421 - val_mae: 2.5893
Epoch 75/100
21/21 [==============================] - 0s 10ms/step - loss: 6.7730 - mae:
1.8142 - val_loss: 14.4568 - val_mae: 2.5749
Epoch 76/100
21/21 [==============================] - 0s 12ms/step - loss: 6.7878 - mae:
1.8243 - val_loss: 15.1749 - val_mae: 2.5955
Epoch 77/100
21/21 [==============================] - 0s 10ms/step - loss: 6.6566 - mae:
1.8063 - val_loss: 14.6603 - val_mae: 2.6040
Epoch 78/100
21/21 [==============================] - 0s 15ms/step - loss: 6.6806 - mae:
1.7971 - val_loss: 14.3121 - val_mae: 2.5426
Epoch 79/100
21/21 [==============================] - 0s 10ms/step - loss: 6.5905 - mae:
1.7778 - val_loss: 14.8108 - val_mae: 2.5616
Epoch 80/100
21/21 [==============================] - 0s 9ms/step - loss: 6.5807 - mae:
1.8057 - val_loss: 14.3868 - val_mae: 2.5512
Epoch 81/100
21/21 [==============================] - 0s 10ms/step - loss: 6.5763 - mae:
1.7712 - val_loss: 14.5696 - val_mae: 2.5595
Epoch 82/100
21/21 [==============================] - 0s 11ms/step - loss: 6.5278 - mae:
```

```
1.7942 - val_loss: 14.5337 - val_mae: 2.5726
Epoch 83/100
21/21 [==============================] - 0s 9ms/step - loss: 6.7557 - mae:
1.8087 - val_loss: 15.1577 - val_mae: 2.6206
Epoch 84/100
21/21 [==============================] - 0s 13ms/step - loss: 6.5155 - mae:
1.7848 - val_loss: 14.2432 - val_mae: 2.5462
Epoch 85/100
21/21 [==============================] - 0s 11ms/step - loss: 6.2791 - mae:
1.7519 - val_loss: 14.7237 - val_mae: 2.5665
Epoch 86/100
21/21 [==============================] - 0s 9ms/step - loss: 6.4072 - mae:
1.7580 - val_loss: 14.4653 - val_mae: 2.5523
Epoch 87/100
21/21 [==============================] - 0s 9ms/step - loss: 6.3527 - mae:
1.7547 - val_loss: 14.9066 - val_mae: 2.5573
Epoch 88/100
21/21 [==============================] - 0s 10ms/step - loss: 6.2020 - mae:
1.7479 - val_loss: 14.4524 - val_mae: 2.5659
Epoch 89/100
21/21 [==============================] - 0s 10ms/step - loss: 6.1660 - mae:
1.7301 - val_loss: 14.8838 - val_mae: 2.5463
Epoch 90/100
21/21 [==============================] - 0s 10ms/step - loss: 6.0187 - mae:
1.7009 - val_loss: 14.3715 - val_mae: 2.5529
Epoch 91/100
21/21 [==============================] - 0s 9ms/step - loss: 6.0471 - mae:
1.6992 - val_loss: 15.2585 - val_mae: 2.5737
Epoch 92/100
21/21 [==============================] - 0s 9ms/step - loss: 6.0718 - mae:
1.7207 - val_loss: 14.9310 - val_mae: 2.5316
Epoch 93/100
21/21 [==============================] - 0s 9ms/step - loss: 6.0825 - mae:
1.6926 - val_loss: 15.3256 - val_mae: 2.5762
Epoch 94/100
21/21 [==============================] - 0s 9ms/step - loss: 5.9756 - mae:
1.7151 - val_loss: 14.5701 - val_mae: 2.5472
```

```
Epoch 95/100
21/21 [==============================] - 0s 10ms/step - loss: 5.9433 - mae:
1.6728 - val_loss: 14.9864 - val_mae: 2.5334
Epoch 96/100
21/21 [==============================] - 0s 9ms/step - loss: 5.9878 - mae:
1.7035 - val_loss: 15.6965 - val_mae: 2.5860
Epoch 97/100
21/21 [==============================] - 0s 9ms/step - loss: 5.9139 - mae:
1.6681 - val_loss: 15.0024 - val_mae: 2.5346
Epoch 98/100
21/21 [==============================] - 0s 9ms/step - loss: 5.9835 - mae:
1.7423 - val_loss: 14.6977 - val_mae: 2.5524
Epoch 99/100
21/21 [==============================] - 0s 9ms/step - loss: 5.6293 - mae:
1.6346 - val_loss: 15.6068 - val_mae: 2.5296
Epoch 100/100
21/21 [==============================] - 0s 6ms/step - loss: 5.9283 - mae:
1.7286 - val_loss: 15.3892 - val_mae: 2.6136
```

测试集 MSE: 22.4138

测试集 MAE: 2.8485 (约 $2848)



示例房屋预测:

真实价格：$18600
预测价格：$19996
误差：$1396

0.2.1 提问点：

1. 为什么要用 StandardScaler 对数据进行标准化？ 答：Boston 房价属于较大的输入数据，而归一化到可以**减小输入数据的尺度**。**消除量纲的影响**，不同特征的量纲可能不同，它们的数值范围可能相差很大。这种量纲差异会导致某些算法（如基于距离的算法）在计算时**产生偏差**。将数据标准化后，所有特征的**尺度一致**，梯度下降过程**更加平稳**，权重更新更加**均匀**，从而加速了算法的收敛速度。

2. 输出层为什么没有激活函数？若添加 sigmoid 会发生什么？ 答：1. 因为 Boston 房价是回归问题，不是分类问题，因此输出层不需要激活函数，回归问题的目标是预测连续的数值，而不是将输出限制在某个特定的范围内。激活函数的**主要作用是引入非线性**，使得神经网络能够学习复杂的函数映射。2. 如果添加 sigmoid 函数，因为这个函数输出值是（0,1），模型的输出将被限制在 (0, 1) 之间。这显然不适合预测房价这种可以取任意实数值的目标。

3. 第二、三层使用 relu 激活函数的意义是什么？ 答：ReLu 函数相比其他函数**不会出现梯度消失**的问题，ReLu 这种简单的**非线性**操作能够有效地打破线性模型的限制，使神经网络能够拟合**更复杂**的函数关系。

4. 解释 mse（均方误差）和 mae（平均绝对误差）的区别。 答：1. mse 是所有预测值与真实值之间差的平方的平均值，mse 对**大误差的惩罚**比 mae 更大。这使得 mse 在存在异常值时可能会被拉高，但也能更好地反映模型在大误差情况下的表现。mse 是一个**可导**的函数，这使得它在使用梯度下降等优化算法时非常方便。2. mae 的单位与**原单位相同**，这使得它在解释上更直观。

0.3 题目三：全连接网络结构对结果的影响分析题

任务描述：

假设某同学设计了一个全连接网络，尝试不同层数和节点数，得到如下实验结果：

模型结构	训练集准确率	测试集准确率
1 层（64 节点）	95%	92%
2 层（128→64 节点）	98%	94%

模型结构	训练集准确率	测试集准确率
4 层（256→128→64→32）	99.9%	88%

0.3.1 提问点：

1. 为什么层数增加时，训练集和测试集准确率差距变大？如何缓解这种现象？

2. 节点数过多可能导致什么问题？节点数过少呢？

3. 如何根据数据规模选择网络层数和节点数？ 1. 答：随着网络层数的增加，模型的**复杂度**也随之增加。这使得模型在训练集上能够学习到**更多的细节和噪声**，从而获得更高的**训练准确率**。然而，这些细节和噪声可能并**不适用于测试集**，导致测试准确率下降，从而出现训练集和测试集准确率差距变大的现象，出现**过拟合**现象。

2. 答：节点指的是神经网络中每一层的**神经元数量**，在全连接层中，节点数决定了该层的**输出维度**。节点数太多会出现**过拟合**，模型过于复杂，能够学习到训练数据中的噪声和细节，导致在训练集上表现很好，但在测试集或新数据上表现不佳，**计算成本高**，更多的神经元意味着更多的参数，这会增加模型的计**算复杂度和训练时间**，还可能导致**特征冗余**，即多个节点提取相似的特征，降低模型的效率。节点数太少会出现欠拟合，模型过于简单，无法捕捉数据中的复杂模式和关系，导致在训练集和测试集上的表现都不佳，限制了模型的性能和泛化能力。

3. 答：**输入层**节点数应与特征数量相匹配，**输出层**节点数应与目标变量数量相匹配。对于**复杂任务**，可能需要更多的层和节点来捕捉数据的复杂模式。更多的层和节点意味着更高的计算成本和更长的训练时间，考虑到计算资源的限制，需要在模型性能和计算成本之间做出权衡。**小规模数据集**：通常选择较浅的网络结构，如 1 到 3 层隐藏层，节点数可以适中，避免过多，可以参考输入层节点数的 75% 左右作为隐藏层节点数的初始值。**中等规模数据集**：可以尝试 3 到 5 层隐藏层，节点数可以稍多一些。可以使用输入层节点数的平方根作为隐藏层节点数的初始值。**大规模数据集**：可以使用较深的网络结构，如 5 层以上，节点数可以更多，可以使用输入层节点数的 1.5 到 2 倍作为隐藏层节点数的初始值。

[ ]: